

1. Reset and Initialization

1.1. Reset behavior

- Apply `rst_n=0` asynchronously.
- Check that internal routing table is empty.
- Outputs (`sout0`, `sout1`) remain at idle (1).

2. Programming Mode Tests

2.1. Basic programming

- Program several address → port mappings:
 - `0x10`→`port0`
 - `0x11`→`port1`
- Verify that both outputs remain idle during programming.

2.2. Overwrite existing mapping

- Reprogram the same address (`0x10`) to a different port.
- Verify that new mapping overrides the old one.

2.3. All-address programming

- Program all 128 (or 256) possible addresses.
- Verify correct storage in the routing table.

3. Functional Mode Tests (Forwarding)

3.1. Basic forwarding

- Program address 0x55→port1.
- Switch to functional mode.
- Send packet [0x55, 0xAA].
- Verify bytes appear on sout1 with correct UART framing.
- Verify sout0 remains idle.

3.2. Routing correctness

- Program multiple addresses to different ports.
- Send data to each and verify output port matches routing table.

3.3. Unmapped address

- Send [0x99, 0x55] when 0x99 not programmed.
- Expected: No data on either output (packet ignored).

3.4. Back-to-back packets

- Send several packets consecutively with minimal gap (1 stop bit time).
- Verify that all are correctly forwarded without loss or overlap.

3.5. Full routing table stress test

- Program full address space (128–256 entries).
- Send random sequence of addresses and data; verify outputs match table.

4. UART Framing and Timing Tests

4.1. Valid frame structure

- Check that receiver correctly decodes start/data/parity/stop bits.

4.2. Invalid start bit

- Send a frame starting with 1 instead of 0.
- Expected: frame ignored, outputs idle.

4.3. Invalid stop bit

- Force stop bit low (0).
- Expected: frame ignored.

4.4. Invalid parity

- Send frame with incorrect even parity.
- Expected: frame ignored.

4.5. Parity correctness

- Send both even and odd data patterns (e.g. 0x00, 0xFF, 0x55, 0xAA) to ensure parity generator/checker behaves correctly.

6. Mode Switching Tests

6.1. Change mode mid-transmission

- Switch `prog` from $1 \rightarrow 0$ (or $0 \rightarrow 1$) during packet.
- Verify that incomplete packet is ignored.

6.2. Reset while forwarding

- Assert `rst_n = 0` mid-transfer; ensure output returns to idle immediately.

7. Boundary and Corner Cases

7.1. Lowest and highest address values

- Program and test forwarding for `addr=0x00` and `addr=0xFF`.

7.2. Data edge patterns

- Test with data `0x00`, `0xFF`, `0x55`, `0xAA`, `0x01`, `0x80`.

7.3. Alternating packets

- Alternate between packets for `port0` and `port1` rapidly to check routing switching.

9. Code and Functional Coverage Goals

- **Functional coverage points:**
 - All address values used at least once.
 - Both output ports used.
 - All error types triggered and ignored.